

Microsoft Q# & Azure Quantum Quick Cheatsheet

A concise reference guide for modern Microsoft Q# (QDK 1.0+) syntax, quantum gates, data structures, and Azure Quantum CLI commands.

1. Q# Core Syntax & Declarations

Functions vs. Operations

- **function** : Pure classical computation with no quantum side effects (deterministic, cannot allocate or modify qubits).
- **operation** : Quantum computation (can allocate qubits, apply gates, perform measurements).

```
// Pure Classical Function
function AddNumbers(a : Int, b : Int) : Int {
    return a + b;
}

// Quantum Operation
operation PrepareSuperposition(q : Qubit) : Unit is Adj + Ctl {
    H(q);
}
```

Qubit Allocation & Release

```
// Single Qubit
use q = Qubit();
H(q);
let result = M(q);
Reset(q); // MUST reset to |0> before release!

// Multi-Qubit Register
use register = Qubit[4];
for q in register {
    H(q);
}
ResetAll(register);
```

2. Standard Quantum Gates

Gate	Q# Code	Matrix Transformation	Description
Pauli X	<code>X(q)</code>	$\begin{pmatrix} 0 & 1 & 1 & 0 \end{pmatrix}$	Bit-flip (\$
Pauli Y	<code>Y(q)</code>	$\begin{pmatrix} 0 & -i & i & 0 \end{pmatrix}$	Combined bit-flip & phase-flip
Pauli Z	<code>Z(q)</code>	$\begin{pmatrix} 1 & 0 & 0 & -1 \end{pmatrix}$	Phase-flip (\$
Hadamard	<code>H(q)</code>	$\frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 & 1 & -1 \end{pmatrix}$	Creates equal superposition (\$
CNOT	<code>CNOT(c, t)</code>	4×4 Unitary	Flips target t iff control $c =$
Controlled-Z	<code>CZ(c, t)</code>	4×4 Unitary	Applies Z on t iff $c =$
Multi-Controlled	<code>Controlled Z([c1, c2], t)</code>	Multi-qubit	Applies gate iff all control qubits are \$

3. Measurement & Diagnostics

```
// Standard Z-basis measurement (returns Result.Zero or Result.One)
let m = M(q);

// Multi-qubit register measurement
mutable results = [];
for q in register {
    set results += [M(q)];
}

// State Vector Diagnostics (Simulator only)
import Microsoft.Quantum.Diagnostics.*;
DumpMachine();
```

4. Control Flow & Classical Integration

```
// Immutable binding
let n = 4;

// Mutable variable
mutable counter = 0;
set counter = counter + 1;

// Conditionals
if m == One {
    X(target);
} elif m == Zero {
    Z(target);
} else {
    // default
}

// Loops
for idx in 0 .. n - 1 {
    Message($"Index: {idx}");
}
```

5. Azure Quantum CLI Quick Reference

```
# Connect to Azure Quantum Workspace
az quantum workspace set -g <resource-group> -w <workspace-name> -l <location>

# List available quantum hardware targets (IonQ, Quantinuum, Rigetti)
az quantum target list -o table

# Submit Q# job to Azure Quantum simulator
az quantum job submit --target-id "microsoft.simulator.fullstate"

# Estimate physical resources (Qubits, T-Gates, runtime)
az quantum job submit --target-id "microsoft.estimator"
```